

多Agent协作办公系统的工作流选型与最小可行架构报告

执行摘要

对于“自己搭建一个多Agent协作办公场景系统，并先从最基本的 workflow 创建开始”的目标，最重要的结论是：**不要把 Agent 框架本身当成工作流引擎**。更稳妥的做法是把系统分成两层：上层是**工作流控制平面**，负责状态、审批、计时器、重试、补偿、审计与权限；下层是**Agent 执行平面**，负责推理、检索、摘要、任务拆解与工具调用。LangGraph 官方文档明确区分了“workflow 的预定路径”与“agent 的动态决策”；Temporal、Camunda、Step Functions 等则提供持久化状态、可恢复执行与编排能力，这正是办公流程上线所需的基础设施。¹

在当前约束大量未指定的前提下，如果你的场景重点是**文档审批、人工参与、任务分配、可视化流程、SLA、审计追踪**，我更推荐把**Camunda 8**作为首个 workflow 引擎：它原生支持 BPMN、用户任务、Tasklist、Webhook/消息/定时触发，并且 2025–2026 年已把“AI agents + humans + systems”的编排作为核心方向。相对地，如果你的重点是**代码优先、长时运行、复杂补偿、微服务协调、低延迟交互**，则更适合选择**Temporal**。如果只是想用最短时间做出原型或内部演示，**n8n**会更快；如果团队是 Python 为主且希望保留高度动态控制流，**Prefect 3** 是一条很值得考虑的中间路线。²

如果必须给出一个“**从现在开始搭建**”的明确建议，我会这样落地：

- **办公流、人机协同优先：** Camunda 8 + PostgreSQL/OpenSearch 或 Elasticsearch + Job Workers + Keycloak/OIDC + OpenTelemetry/Prometheus
- **工程流、代码弹性优先：** Temporal + PostgreSQL/MySQL + Worker SDK + OTel + Keycloak/OIDC
- **快速原型优先：** n8n + PostgreSQL + Redis(queue mode) + 独立的 Agent API 服务
- **AWS 全栈托管优先：** AWS Step Functions + EventBridge + Lambda/ECS + Bedrock

这些建议的依据，是各引擎在编程模型、状态持久化、触发器、人工审批、弹性恢复、部署复杂度与 AI 集成路径上的官方能力边界。需要特别说明的是，各产品**没有统一的官方横向 benchmark**，所以文中关于“延迟/吞吐倾向”的结论是**基于架构与文档能力的定性判断**，不是同一测试环境下的跑分。³

目标与约束界定

本报告面向的**明确目标**是：搭建一个用于“多Agent协作办公”的系统，并且先从最基础的 **workflow 创建与工作流引擎选型**开始。结合你给出的范围，系统至少会覆盖文档接入、规则判断、人工审批、任务分发、信息汇总、通知归档这类典型办公链路。由于这类场景通常包含**长时运行、人工介入、外部系统调用与异常补救**，因此工作流引擎需要具备比单纯 Agent 框架更强的持久化、审计、权限与恢复能力。⁴

下表把“已明确目标”和“仍未指定约束”拆开列出，便于后续选型不混淆。

维度	当前状态	说明
系统目标	已指定	多Agent 协作办公
第一步目标	已指定	从最基本的 workflow 创建开始

维度	当前状态	说明
示例业务流	已指定	若未另行指定，按“文档审批 + 任务分配 + 信息汇总”假设
并发规模	未指定	峰值 workflow 实例数、同时运行 Agent 数未指定
延迟目标	未指定	未指定是否要求秒级交互、亚秒级响应或分钟级批处理
云端 / 本地	未指定	未指定必须公有云、私有云、混合云或纯本地部署
预算	未指定	未指定许可预算、算力预算、运维预算
语言偏好	未指定	系统业务语言偏好未指定 ；仅“本报告输出语言”为中文
团队技术偏好	未指定	未指定偏 Python、Java、TypeScript、Go 还是低代码
人工审批占比	未指定	未指定人机协同比例
合规要求	未指定	未指定数据驻留、隐私、审计、行业监管要求
外部集成范围	未指定	未指定接入邮件、IM、OA、CRM、文档库、知识库的范围
SSO / 权限模型	未指定	未指定 OIDC/SAML/LDAP、RBAC/ABAC 等要求

这组“未指定”非常关键，因为它几乎直接决定 workflow 引擎的首选项：如果后续确认是**高人机协同 + 可视化治理**，Camunda 会明显领先；如果确认是**高并发服务协调 + 长时可可靠执行**，Temporal 会更合适；如果是**轻量自动化 + 内部工具编排**，n8n 可能最省成本；如果是 **Python 团队 + 动态 agent 控制流**，Prefect 会很顺手。 5

workflow 引擎候选对比

先给结论：对“多 Agent 协作办公”这个特定问题，最值得优先评估的是 **Camunda 8、Temporal、Prefect、n8n**；Airflow 与 Argo 更偏批处理/容器任务；Cadence 适合熟悉其生态的团队；Step Functions 在 AWS 原生环境里极具性价比。下面两张表分别从“运行模型”和“工程生态”两个角度做对比。表中“低/中/高”“强/中/弱”属于**定性综合判断**，依据的是官方文档、官方博客、发布说明与项目主页，不代表统一压测结果。 6

引擎	编程模型	状态管理与持久化	可扩展性	延迟/吞吐倾向	调度与触发器
Temporal 7	代码 workflow；Workflow/Activity；子 workflow、Signals、Queries、Updates	服务端持久化事件与执行状态；Visibility 独立存储	高；Worker 可水平扩展并可调优	交互与长时流程都强 ；偏低延迟控制流	Schedules、Client/API、Signals、子 workflow 流唤起
Cadence 8	代码 workflow；Go/Java 客户端成熟	服务端持久化；高级可见性可接 Elasticsearch	高	与 Temporal 接近 ，但生态相对小	Cron、Signals、Queries、Child Workflows

引擎	编程模型	状态管理与持久化	可扩展性		延迟/吞吐倾向	调度与触发器	
Camunda 8 / Zeebe 9	BPMN/DMN + Job Workers; 流程图与代码结合	Zeebe 事件流 + 分区复制; 配套 Operate/ Tasklist/Identity	高; 分区与复制扩展		办公流程与人机协同强 ; 吞吐依赖分区设计	Timer、Message、Webhook Connector、User Task、API	
Prefect 3 10	纯 Python flow/task; 动态控制流, 不需预编译 DAG	自动状态跟踪; 结果可持久化到远端存储; 事件/自动化	中高		动态 AI/ Python 流程强	Schedules、Automations、Custom Events、Webhook (Cloud)	
n8n 11	可视化节点流; 低代码/脚本混合	默认 SQLite; 生产建议 PostgreSQL; queue mode 用 Redis	中; queue mode 可扩展, 但架构有边界		原型快, 事务级复杂流一般	Webhook、Cron、App Triggers、Error Trigger	
Argo Workflows 12	Kubernetes CRD; DAG/ steps; 容器原生	Workflow 先存 K8s/etcd; 大工作流可 offload 到 SQL; 可归档	高, 前提是你已有成熟 K8s		并行容器吞吐强 ; 交互办公流不占优	Argo Events、Webhook、Cron、消息队列、API	
Apache Airflow 13	Python DAG; 任务/依赖/调度中心化	Metadata DB (SQLAlchemy 后端)	高; 模块化、消息队列和 worker 扩展		批处理最强 ; 低延迟/长等待人审不理想	Schedule、REST/ API、Sensors、Datasets、事件调度	
AWS Step Functions 14	Amazon States Language; 可视化状态机	托管式状态与执行历史	很高 (托管)		Standard 适合长时审计; Express 适合高吞吐事件流	API/EventBridge/ 服务集成/HTTPS	
引擎	可视化与监控	事务 / 补偿	分布式协调	部署复杂度	社区活跃度	许可证	AI/LLM 集成难易度
Temporal 15	Temporal UI、Visibility、Worker 性能指标	Saga/补偿模式成熟, 但需要你在代码里明确建模	强; 适合服务编排	中高; 自托管需持久化与安全配置	高	MIT 16	高; 官方已有 AI SDK 集成入口与 durable agent 叙事 17

引擎	可视化与监控	事务 / 补偿	分布式协调	部署复杂度	社区活跃度	许可证	AI/LLM 集成难易度
Cadence 18	Cadence Web、搜索可见性	有 Saga 类与补偿模式	强	中高	中	Apache-2.0 19	中；可做，但缺少当前主流 AI 集成叙事
Camunda 8 / Zeebe 20	Modeler、Operate、Tasklist、Identity 完整	BPMN 补偿、事件、人工任务、异常治理都成熟	强；分区领导者/副本机制	中高；生产推荐 K8s + Helm	高	Camunda License v1；SaaS/自管并存 21	高；官方已明确支持 agentic orchestration
Prefect 3 22	实时监控、事件、自动化	Prefect 3 强调 transactional semantics 与幂等性	中高	中	高	Apache-2.0 23	高；官方直接面向 AI agents 与 Pydantic AI 24
n8n 25	画布直观，错误 workflow 清晰	重试/错误 workflow 有，但没有强事务级补偿语义	中	低到中；单机简单，queue mode 生产化后上升	高	Sustainable Use License；fair-code，非 OSI 开源 26	高；原生 AI Agent/Tool 节点
Argo Workflows 27	Argo Server 提供 UI/API，支持 archive/logs	retry/exit handlers 强；补偿通常需你自行设计	中高；依附 K8s 控制面	高；如果没有 K8s 能力会很重	高	Apache-2.0 28	中；适合把 Agent 打包成容器任务
Apache Airflow 29	UI、日志、Grid/Gantt、指标成熟	retry/trigger rules/callback 强；分布式补偿并非核心模型	中	中高	很高	Apache-2.0 30	中高；2026 已有官方 Common AI Provider
AWS Step Functions 31	控制台、CloudWatch、X-Ray 成熟	Retry/Catch 强；补偿以状态机设计实现	强（托管）	低（对用户）	高（商业生态）	商业托管服务	中高；对 AWS/Bedrock 最顺手 32

综合这些能力，**对你的场景的推荐排序**可以写得很明确：

第一档：Camunda 8、Temporal

一个偏人机协同与可视化治理，一个偏代码编排与持久执行。如果你想要“办公系统感”、审批感、SLA 感、审计感，先看 Camunda；如果你想要“后端平台感”、服务可靠性、复杂补偿和 SDK 体验，先看 Temporal。

33

第二档：Prefect、n8n

Prefect 适合 Python 团队和动态 AI 流；n8n 适合尽快做出业务原型和连接大量 SaaS/API，但不建议一开始就把它当成重治理、重审计的主 workflow 底座。

34

第三档：Argo、Airflow、Cadence、Step Functions

Argo 和 Airflow 更适合容器/数据/批处理背景明显的团队；Cadence 更适合已有经验的组织；Step Functions 则是 AWS 全栈组织的强力选项，但云锁定更强。

35

多Agent协作架构与通信模式

对多Agent系统来说，“谁负责决定下一步”比“Agent 用什么模型”更重要。LangGraph 文档把 workflow 与 agent 区分得很清楚：workflow 更适合确定路径，agent 更适合动态决策；而多 Agent 时，又可以采用 supervisor、swarm、subgraph 等模式。AutoGen 则把多代理对话当成可编程基础设施。把这些模式与传统消息系统结合起来，适合办公场景的主流架构大致有六种。

36

架构模式	协调方式	常用协议 / 消息格式	一致性与恢复	权限与安全	适用判断
集中式协调者 37	一个 supervisor 决定调用哪个 agent	内部函数调用、HTTP/gRPC；JSON 消息	状态集中，最容易审计与回放；恢复逻辑简单	最容易做 RBAC、审计与人工插入	最适合办公起步版
事件驱动 / 消息总线 38	Producer 发布事件，多个 agent 订阅	Kafka、AMQP、NATS；JSON/CloudEvents	一般是最终一致；可重放、DLQ、订阅解耦	topic/queue ACL、签名、租户隔离关键	适合高解耦、多系统集成
P2P / Swarm 39	agent 之间直接 handoff	对话消息、HTTP/gRPC	自主性高，但全局状态难控；恢复难度高	容易发生权限扩散与工具误用	适合探索任务，不适合审批主流
黑板模式 40	共享黑板 / 共享状态仓，agent 竞争式贡献	共享文档、KV、向量库、append-only 日志	取决于黑板存储；适合并行发现与渐进汇总	必须做字段级/文档级权限控制	适合信息发现、汇总、协同推理
工作流驱动 Agent 41	工作流引擎决定顺序，agent 只是任务执行者	引擎任务队列、gRPC、Webhook	最强的业务可恢复性；重试、补偿、SLA 明确	权限可绑定到流程节点与 worker 身份	生产化首选

架构模式	协调方式	常用协议 / 消息格式	一致性与恢复	权限与安全	适用判断
微服务 + Orchestrator 42	每个 agent 是独立服务，orchestrator 串联	HTTP/gRPC + MQ; JSON/ Protobuf	介于集中式和事件式之间	需要统一身份、限流、观察性	适合大团队分工明确的组织

对办公场景，我建议优先采用“**工作流驱动 Agent + 集中式协调者**”的混合模式。更具体地说：**工作流引擎**负责“流程合法性”；**Supervisor Agent**负责“智能判断与动态路由”；**各专业 Agent**负责“执行具体认知/工具动作”；**消息总线**只作为跨系统解耦和异步通知，不要一开始就让它承担主状态机职责。这样做的好处是，审批、定时器、重试、补偿、审计、权限都还在可控的 workflow 层，而 Agent 的不确定性被限制在可观测、可重放的局部节点中。 43

在消息设计上，建议统一层**Agent Envelope**，哪怕底层用 HTTP、gRPC、Kafka 还是引擎内部任务队列，都至少携带这些字段：workflow_id、instance_id、step_id、agent_role、trace_id、idempotency_key、deadline_ms、input_ref、auth_context、retry_count、schema_version。这样做的目的不是“格式优雅”，而是为了把**可追踪性、幂等性、权限、协议版本**提前固化下来；否则随着 Agent 数量上升，排障会迅速恶化。Temporal signals/queries、Cadence signals、Camunda user tasks / connectors、Step Functions state data 其实都在强调同一件事：**流程状态必须是可寻址、可检查、可恢复的对象**。 44

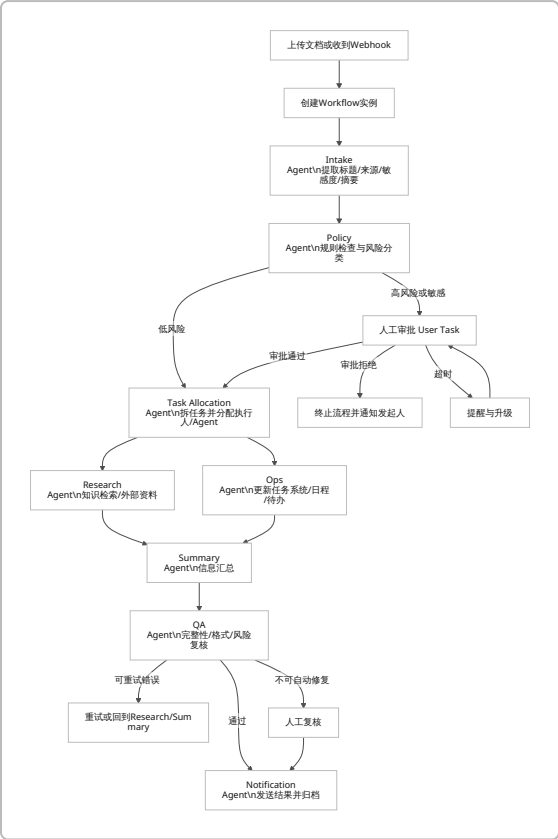
一致性方面，不建议追求“全链路强一致”。对这类系统，更现实的方案是：**工作流引擎内部状态强可恢复；外部副作用通过幂等键、Outbox/Inbox、补偿动作来保证最终业务一致**。AWS 对 saga 的解释、Temporal/Cadence 对 saga/compensation 的支持、Step Functions 的 Retry/Catch，都说明生产系统的关键不是“永不失败”，而是“失败后可以安全继续或安全回退”。 45

安全上，推荐最小集是：**人类用户走 OIDC/SSO；Agent/Worker 到引擎走 mTLS 或最少 scoped API key；敏感文档加密存储；Prompt 与附件分级脱敏；工具权限最小化；所有跨系统调用保留审计轨迹**。Temporal 文档明确支持 mTLS 与客户端侧数据转换/加密；Camunda 提供 Identity / Admin / Tasklist 授权链路；n8n 提供 SSL、SSO、2FA、加密密钥轮换；Step Functions 依赖 IAM、TLS 与 KMS。 46

最小可行工作流原型

按照你的要求，在未指定业务流时，这里采用“**文档审批 + 任务分配 + 信息汇总**”作为最小可行原型。这个原型的目标不是“一步到位变成全能办公助手”，而是验证以下四件事：

能否稳定创建 workflow 实例；能否插入人工审批；能否把不同 Agent 协同串起来；能否在失败时恢复或补偿。这四点正对应了 Camunda 的 user task / webhook / tasklist 能力，Temporal 的 signals / child workflows / schedules 能力，以及 workflow/agent 分层原则。 47



如果你把 **Camunda 8** 作为第一版 workflow，图里的 **人工审批 User Task** 可以直接落到 Tasklist；
上传文档或收到Webhook 可以用 inbound connector / HTTP webhook 触发；**Task Allocation Agent**、**Research Agent**、**Summary Agent** 则作为 job workers 或外部服务。若换成 **Temporal**，则 **人工审批** 会变成“工作流阻塞等待 signal + 你自己做一个审批 UI”；优点是全链路代码可控，代价是需要自行补齐 Tasklist 这类人工作台。 48

下面给出一个建议的 Agent 划分。这里的设计原则是：**把 LLM 真正擅长的部分放进 Agent，把流程确定性和副作用控制留给 workflow。**

Agent / 角色	职责	输入	输出	触发条件	失败处理
Intake Agent	提取文档元数据、初始摘要、敏感度候选标签	原始文档、上传人、上下文	结构化元数据、初始摘要	工作流启动后立即触发	文档解析失败则转人工；OCR/解析类错误限制重试次数
Policy Agent	根据规则和提示词判断是否需要人工审批	文档元数据、规则配置	风险级别、审批建议、路由标签	Intake 完成后触发	保守失败：默认进入人工审批
Human Approver	审批/拒绝/要求补充材料	文档摘要、风险理由、附件	审批意见、批注	高风险分支	超时提醒、升级到上级或共享池

Agent / 角色	职责	输入	输出	触发条件	失败处理
Task Allocation Agent	把后续动作拆解成 research / ops / summary 子任务	审批结果、流程上下文	子任务列表、负责人/Agent 映射	审批通过或低风险自动通过	失败时回退到固定模板分派
Research Agent	查知识库、外部资料、内部历史记录	子任务、关键词、知识源	资料清单、事实片段、引用来源	被指派到信息收集任务时	外部源失败重试；超过阈值降级为仅内部知识
Summary Agent	把分散信息合成为可读产出	资料清单、任务结果	周报/简报/审批意见草稿	Research 完成后	生成质量不足时由 QA 触发返工
QA Agent	检查完整性、引用、格式、敏感信息	汇总结果、质量规则	通过/驳回、缺陷列表	Summary 输出后	可修复则退回 Summary；不可修复则人工复核
Notification Agent	发邮件/IM/归档到知识库/更新工单	最终结果、收件人列表	发送状态、归档 ID	QA 通过或人工复核完成后	外部通知失败重试并保留 outbox

这个 MVP 的核心不在“Agent 多”，而在**流程骨架正确**。第一版建议只做 4-6 个关键 Agent，不要一上来就做 supervisor of supervisors。LangGraph 的 supervisor / subgraph 适合在第二阶段引入，用来封装复杂的认知子流程；第一阶段更重要的是把人工审批、重试、审计、归档跑通。⁴⁹

技术栈与选型建议

如果把“你现在就要定一个 workflow 底座”这个问题说得更直接，我的建议是：**默认选 Camunda 8；如果团队明显是工程后端导向而不是流程治理导向，则改选 Temporal**。理由很简单：你给出的办公示例天然带有人审、文档、例外处理与 SLA，这些都属于 Camunda 的强项；而 Temporal 更像“可靠执行内核”，非常强，但人工作台与 BPMN 可视化能力需要你自己补。⁵⁰

下面按优先级场景给出更具体的推荐组合。

优先级场景	推荐 workflow 引擎	推荐配套组件	推荐理由
低延迟高并发	Temporal ⁵¹	PostgreSQL/MySQL；可选 Kafka/NATS 作为异步事件层；FastAPI/gRPC Agent 服务；OTel + Prometheus/Grafana；Keycloak/OIDC；如已在 K8s 中可接 Linkerd/Istio	持久执行、signals、child workflows、worker 调优与长时运行都很强，适合复杂服务协调

优先级场景	推荐 workflow 引擎	推荐配套组件	推荐理由
易开发快速迭代	n8n 或 Prefect ⁵²	n8n 走 PostgreSQL + Redis(queue mode); Prefect 走 Python + 远端 result storage; 独立 Agent API; 基础监控与告警	n8n 画布上手极快; Prefect 对 Python/AI 团队更自然, 动态控制流更强
完全开源自托管	Temporal 或 Prefect; K8s/批处理团队可考虑 Argo ⁵³	Postgres; OpenTelemetry; Prometheus/Grafana; Vault/Keycloak; Argo 场景需成熟 Kubernetes	Camunda 8 与 n8n 都有许可边界; 如果你要求 OSI 意义上的开源, 自托管首选应避免它们
云托管优先	Temporal Cloud、Camunda SaaS, AWS 场景可用 Step Functions ⁵⁴	托管控制面 + 轻量 Worker; CloudWatch/X-Ray 或厂商观测能力; OIDC/API keys/mTLS	最省运维; 代价是供应商绑定和局部可定制度下降

如果你的目标是**第一阶段尽快上线一个“文档审批 + 任务分配 + 汇总”的办公流**, 我会给出下面这套更聚焦的建议:

推荐组合 A: Camunda 8 作为工作流层

Camunda 8 + PostgreSQL/OpenSearch(或 Elasticsearch) + Job Workers(用 Java/Go/Node/Python 任一) + 独立 Agent API + Keycloak/OIDC + OpenTelemetry。

这套组合最适合有人工审批、表单、异常流程、运营可视化要求的团队。Webhook 触发、用户任务、Tasklist、Operate、身份治理与 agentic orchestration 方向是它的优势。 ⁵⁵

推荐组合 B: Temporal 作为工作流层

Temporal + PostgreSQL + Worker SDK + 自建审批 Web UI + LangGraph/AutoGen 作为某些 Activity 内部的认知子流程 + OTel + OIDC。

这套组合更适合工程团队, 希望把一切流程逻辑写在代码中, 并且强调可靠重试、局部补偿、消息注入、长时暂停恢复。 ⁵⁶

为什么不建议第一版直接以 LangGraph / AutoGen 作为 workflow 本体

因为它们是很好的 **agent orchestration / agent runtime**, 不是完整的**企业工作流控制平面**。LangGraph 非常适合做 Supervisor、Subgraph、Swarm 和持久状态线程; AutoGen 适合多代理对话与协作; 但审批、审计、计时器、SLA、权限、补偿、归档这些办公系统“骨架能力”, 仍然更适合交给 Temporal / Camunda / Step Functions / Prefect 这类 workflow orchestration 工具。 ⁵⁷

开发与部署步骤清单

一个可维护的多Agent办公系统，不应该先从“调 prompt”开始，而应该先从控制平面、契约、测试与观测性开始。官方文档无论是 Temporal、Camunda、Airflow、Argo、Prefect 还是 Step Functions，都在不同角度强调了同一个事实：**生产 workflow 系统首先是状态系统，其次才是任务系统。** ⁵⁸

推荐的落地步骤如下：

1. 先定控制平面，不先定 Agent 框架。

在 README 级别写清楚：工作流引擎是谁、Agent 运行在哪里、消息怎么传、人工审批怎么插入、失败后谁负责补偿。第一天先把“控制平面边界”写出来，比第一天就做智能更重要。 ⁵⁹

2. 建立最小代码结构。

推荐把仓库拆成：

```
/workflow-definitions
/workers
/agents/intake
/agents/policy
/agents/research
/agents/summary
/shared/contracts
/shared/idempotency
/web/approval-ui
/infra/helm-or-compose
/tests/unit
/tests/integration
/tests/e2e
/docs/runbooks
```

其中 `/shared/contracts` 用来放统一消息 schema、trace 字段、错误码与权限标签。

1. 先实现一条“空心流程”再填 Agent。

只做“上传 -> 审批 -> 通知 -> 归档”的空心链路，所有 Agent 用 mock 返回。这样可以先验证触发器、审批、状态持久化、重试与 UI 路径。等 workflow 骨架稳定后，再给 Agent 注入真实能力。

2. 把每个 Agent 都做成可替换的任务节点。

不要把多个推理步骤糊在一个“大总管 Agent”里。每个 Agent 节点都必须有：输入 schema、输出 schema、超时、重试策略、幂等策略、降级策略、审计标签。

3. 测试分三层，不可以省。

单元测试关注：节点逻辑、路由规则、schema 校验。

集成测试关注：工作流引擎、数据库、消息总线、外部 API mock 的互动。

端到端测试关注：真实文档、真实人工审批、真实通知、失败重试、补偿回滚。

如果选 Temporal 或 Camunda，还应加入“暂停后恢复”“重复触发”“外部 signal/approval 重放”类场景。 ⁶⁰

4. CI/CD 以“流程版本化”而不是“仅代码版本化”为主。

Workflow 定义、消息 schema、Prompt 模板、工具白名单都应进入版本控制。上线时要有迁移策略，

避免“新版本 worker 处理旧流程实例”造成语义破坏。Temporal、Cadence、Camunda 都非常依赖版本兼容意识。 61

5. 观测性要覆盖 workflow 与 agent 两层。

最少要有：

`workflow_id / step_id / agent_role / attempt / latency / token_usage / external_call / human_wait_time / queue_backlog / dead_letter_count`。

对用户来说，“为什么没完成”必须能从 workflow 和 trace 两个视角同时看见。Airflow、Step Functions、Temporal、Camunda 都把 UI/metrics/logs 当成核心能力而不是附属能力。 62

6. 回滚不是“回滚代码”，而是“回滚流程影响”。

例如：已经发出的通知如何撤回、已经创建的工单如何关闭、已经写入的审批记录如何标注为 superseded、已经调用的外部系统如何补偿。Saga/compensation 需要在设计阶段就写出来。 63

7. 性能测试必须贴近场景，而不是只测 API QPS。

办公流最容易被忽视的瓶颈通常不是引擎本身，而是：通知风暴、审批积压、外部系统限流、向量检索延迟、LLM 排队、文件解析 CPU 峰值。压测要覆盖：高峰上传、批量审批、重试风暴、消息堆积、外部服务 429/5xx。

8. 上线前准备 runbook。

至少写清楚：实例卡住如何处理、人工补批怎么做、如何强制重试、如何暂停新触发、如何切主/切库、如何查看单实例全链路、如何审计某份文档的所有外部调用。

如果你打算把第一版做得更稳，我建议在 MVP 期就预留两个“以后一定会用到”的能力：**幂等键**和**人工介入节点**。多数办公系统早期不是死在模型效果，而是死在重复执行和异常状态无法手工修复。 64

示例代码与参考资料

这一节不直接贴长代码，而是给出**最有价值的官方文档、示例仓库、论文与中文资料入口**。由于平台太多，这里只列与你的目标最相关的资源，且尽量优先官方材料。引用链接请直接点各条后的引用。 65

类别	资源	用途说明
官方文档	Camunda 8 Docs 66	适合人机协同、BPMN、Tasklist、Self-Managed/SaaS 全链路入门
官方文档	Temporal Docs 67	适合 durable execution、signals、child workflows、self-hosting
官方文档	Prefect v3 Docs 68	适合 Python 工作流、事件自动化、结果持久化
官方文档	n8n Docs 69	适合快速原型、queue mode、AI Agent 节点
官方文档	Argo Workflows / Argo Events Docs 70	适合容器原生并行任务与事件驱动
官方文档	Apache Airflow Docs 71	适合 batch-oriented workflows、监控、UI、事件调度能力

类别	资源	用途说明
官方文档	AWS Step Functions Docs ⁷²	适合 AWS 托管状态机、观测与错误处理
AI 框架文档	LangGraph Workflows and Agents / Multi-agent ⁷³	理解 workflow 与 agent 的边界、supervisor/subgraph 模式
AI 框架文档	AutoGen 官方文档 ⁷⁴	理解事件驱动多代理协作与人机混合模式
样例与指南	Camunda Human Task Guide / User Task Docs ⁷⁵	文档审批、人审插入最相关
样例与指南	Temporal Signals / Child Workflows / Schedules ⁷⁶	适合自己做审批 UI 的工程路线
样例与指南	n8n AI Agent Tool / Error Handling ⁷⁷	快速做出多Agent 原型与错误流
论文	AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation ⁷⁸	多代理协作的经典技术报告
论文	Blackboard Multi-Agent Systems for Information Discovery in Data Science ⁷⁹	黑板模式在 LLM 多代理中的近期代表工作
中文资料	腾讯云智能体开发平台：智能体应用模式 / Multi-Agent 工作流实践 ⁸⁰	中文产品化参考，适合理解国内工作流画布如何组织多Agent
中文资料	阿里云百炼：智能体编排并入工作流应用 ⁸¹	中文产品动态，适合理解“Agent 编排转 workflow 化”的行业方向
中文资料	InfoQ 中文：软件架构与 Agentic AI 趋势 ⁸²	适合从架构师视角理解多Agent 从实验走向治理化系统

如果你问“第一份示例工程应该参考哪类项目”，我的建议是不要直接找“一个包治百病的多Agent demo”，而是分别找三类：一个 **workflow 官方 quickstart**、一个 **human task/approval 示例**、一个 **multi-agent supervisor 示例**。

这三类拼起来，比追一个“大而全 AI 办公助理”模板更容易形成真正可运维的系统。 ⁸³

风险与注意事项

第一个常见陷阱，是把 **Agent 框架误当成 workflow 引擎**。从实验角度看，这样很快；从办公生产角度看，风险极高。原因在于 Agent 框架通常擅长“推理流程”，而不是“业务流程控制”：它们能决定下一步怎么想，但未必擅长长时挂起、人工审批、状态恢复、补偿和合规审计。LangGraph 自己都把 workflow 与 agent 区分开来；Camunda 与 Temporal 则分别从流程治理和 durable execution 的角度补齐了这部分能力。 ⁸⁴

第二个陷阱，是**忽视幂等性与副作用去重**。工作流引擎普遍支持 retries，但如果你的通知、建单、写库、发邮件、更新 OA 没有幂等键，重试就可能造成重复审批、重复消息、重复工单。Step Functions 的 Retry/Catch、Temporal/Cadence 的恢复与 saga、Camunda 的补偿与人审分支，本质上都假设你愿意把外部副作用明确建模。 ⁸⁵

第三个风险，是**许可边界与“完全开源”预期不一致**。如果你要求 OSI 意义上的完全开源自托管，需要特别注意：n8n 使用 Sustainable Use License，属于 fair-code/source-available，而不是 OSI 开源；Camunda 8 的

自管源码从 8.6 起使用 Camunda License v1，生产使用存在许可边界。因此，如果“完全开源”是硬约束，首选应转向 Temporal、Prefect、Argo、Airflow、Cadence 这类更明确的开源许可项目。 86

第四个风险，是**低估运维复杂度**。

- Temporal 自托管不是单二进制玩具，依赖持久化和安全配置，适合有平台能力的团队。
- Camunda 8 生产推荐 Kubernetes/Helm，功能完整但平台面更广。
- Argo 的前提是你已经有成熟 Kubernetes 团队；否则会“为了 workflow 先引入一个平台”。
- Airflow 在批处理世界很成熟，但安装与数据库/调度器维护并不轻。
- n8n 单机很轻，但只要上 queue mode，就进入 PostgreSQL + Redis + worker 拓扑治理。
- Step Functions 用户运维最轻，但云约束最强。 87

第五个风险，是**敏感文档与模型调用的隐私泄露**。办公场景天然包含附件、审批意见、报表摘要、客户信息、内部政策等高敏数据。至少要落实：文档分级、Prompt 脱敏、凭证最小权限、密钥轮换、传输加密、存储加密、审计可追踪。Temporal 支持 mTLS 与客户端数据转换/加密；Camunda 有 Identity / 授权体系；n8n 文档强调 SSL、SSO、2FA 和加密密钥轮换；Step Functions 可与 IAM/KMS/TLS 组合。 88

从扩展性与运维成本角度，最现实的经验判断是：

- **最低起步成本**：n8n、Step Functions
- **最平衡的工程化路径**：Camunda 8、Prefect
- **最强的代码级可靠执行能力**：Temporal
- **最适合已有 K8s / 数据平台的组织**：Argo、Airflow
- **最不建议作为你第一版默认选项的**：P2P/swarm 式“纯 agent 自由协商”

这个排序并不表示能力高低，而是表示与你当前问题“多 Agent 办公场景、先从 workflow 开始”的匹配程度。

89

最后，必须明确几项仍然**未决但会显著影响最终架构**的问题：并发规模、目标延迟、是否必须本地部署、预算、目标系统集成范围、是否强依赖中文业务 UI、是否必须全开源、是否需要企业级 SSO/细粒度权限、以及人工审批在流程中的占比。目前这些都还是“未指定”，因此文中的推荐是面向“典型企业办公多 Agent 系统”的最稳妥默认值，而不是唯一正确答案。若后续这些约束落地，最终选择很可能会从“Camunda vs Temporal”二选一，进一步收敛为一个更清晰的方案。

1 36 43 59 73 84 <https://docs.langchain.com/oss/python/langgraph/workflows-agents>
<https://docs.langchain.com/oss/python/langgraph/workflows-agents>

2 4 47 <https://docs.camunda.io/docs/components/modeler/bpmn/user-tasks/>
<https://docs.camunda.io/docs/components/modeler/bpmn/user-tasks/>

3 <https://docs.camunda.io/docs/self-managed/deployment/helm/install/production/>
<https://docs.camunda.io/docs/self-managed/deployment/helm/install/production/>

5 75 83 <https://docs.camunda.io/docs/guides/orchestrate-human-tasks/>
<https://docs.camunda.io/docs/guides/orchestrate-human-tasks/>

6 13 71 <https://airflow.apache.org/docs/apache-airflow/stable/index.html>
<https://airflow.apache.org/docs/apache-airflow/stable/index.html>

7 41 51 67 <https://docs.temporal.io/>
<https://docs.temporal.io/>

- 8 <https://cadenceworkflow.io/>
<https://cadenceworkflow.io/>
- 9 65 66 <https://docs.camunda.io/>
<https://docs.camunda.io/>
- 10 22 34 68 <https://docs.prefect.io/v3/get-started>
<https://docs.prefect.io/v3/get-started>
- 11 <https://docs.n8n.io/hosting/configuration/supported-databases-settings/>
<https://docs.n8n.io/hosting/configuration/supported-databases-settings/>
- 12 35 70 <https://argoproj.github.io/workflows/>
<https://argoproj.github.io/workflows/>
- 14 <https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>
<https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>
- 15 <https://docs.temporal.io/visibility>
<https://docs.temporal.io/visibility>
- 16 53 <https://github.com/temporalio/temporal/blob/main/LICENSE>
<https://github.com/temporalio/temporal/blob/main/LICENSE>
- 17 <https://docs.temporal.io/develop/typescript>
<https://docs.temporal.io/develop/typescript>
- 18 <https://cadenceworkflow.io/blog/2025/04/11/2025-04-11-announcing-cadence-web-v4/announcing-cadence-web-v4>
<https://cadenceworkflow.io/blog/2025/04/11/2025-04-11-announcing-cadence-web-v4/announcing-cadence-web-v4>
- 19 <https://github.com/cadence-workflow/cadence/blob/master/LICENSE>
<https://github.com/cadence-workflow/cadence/blob/master/LICENSE>
- 20 33 50 <https://camunda.com/platform/tasklist/>
<https://camunda.com/platform/tasklist/>
- 21 <https://docs.camunda.io/docs/reference/licenses/>
<https://docs.camunda.io/docs/reference/licenses/>
- 23 <https://github.com/PrefectHQ/prefect/blob/main/LICENSE>
<https://github.com/PrefectHQ/prefect/blob/main/LICENSE>
- 24 <https://www.prefect.io/solutions/agents>
<https://www.prefect.io/solutions/agents>
- 25 <https://docs.n8n.io/flow-logic/error-handling/>
<https://docs.n8n.io/flow-logic/error-handling/>
- 26 86 <https://docs.n8n.io/sustainable-use-license/>
<https://docs.n8n.io/sustainable-use-license/>
- 27 <https://argo-workflows.readthedocs.io/en/latest/argo-server/>
<https://argo-workflows.readthedocs.io/en/latest/argo-server/>
- 28 <https://github.com/argoproj/argo-workflows/blob/master/LICENSE>
<https://github.com/argoproj/argo-workflows/blob/master/LICENSE>
- 29 <https://airflow.apache.org/docs/apache-airflow/stable/ui.html>
<https://airflow.apache.org/docs/apache-airflow/stable/ui.html>

30 <https://github.com/apache/airflow/blob/main/dev/airflow-license>
<https://github.com/apache/airflow/blob/main/dev/airflow-license>

31 72 <https://docs.aws.amazon.com/step-functions/>
<https://docs.aws.amazon.com/step-functions/>

32 <https://aws.amazon.com/blogs/machine-learning/orchestrate-generative-ai-workflows-with-amazon-bedrock-and-aws-step-functions/>
<https://aws.amazon.com/blogs/machine-learning/orchestrate-generative-ai-workflows-with-amazon-bedrock-and-aws-step-functions/>

37 49 <https://reference.langchain.com/python/langgraph-supervisor>
<https://reference.langchain.com/python/langgraph-supervisor>

38 <https://www.enterpriseintegrationpatterns.com/patterns/messaging/MessageBus.html>
<https://www.enterpriseintegrationpatterns.com/patterns/messaging/MessageBus.html>

39 <https://reference.langchain.com/python/langgraph-swarm>
<https://reference.langchain.com/python/langgraph-swarm>

40 79 <https://research.google/pubs/blackboard-multi-agent-systems-for-information-discovery-in-data-science/>
<https://research.google/pubs/blackboard-multi-agent-systems-for-information-discovery-in-data-science/>

42 <https://arxiv.org/pdf/2204.07210>
<https://arxiv.org/pdf/2204.07210>

44 56 76 <https://docs.temporal.io/sending-messages>
<https://docs.temporal.io/sending-messages>

45 <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga.html>
<https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga.html>

46 88 <https://docs.temporal.io/self-hosted-guide/security>
<https://docs.temporal.io/self-hosted-guide/security>

48 55 <https://docs.camunda.io/docs/components/connectors/protocol/http-webhook/>
<https://docs.camunda.io/docs/components/connectors/protocol/http-webhook/>

52 89 <https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/>
<https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/>

54 <https://temporal.io/>
<https://temporal.io/>

57 <https://docs.langchain.com/oss/python/langgraph/overview>
<https://docs.langchain.com/oss/python/langgraph/overview>

58 <https://docs.temporal.io/temporal-service>
<https://docs.temporal.io/temporal-service>

60 <https://docs.temporal.io/handling-messages>
<https://docs.temporal.io/handling-messages>

61 <https://github.com/temporalio/temporal/releases>
<https://github.com/temporalio/temporal/releases>

- 62 <https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/logging-monitoring/index.html>
<https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/logging-monitoring/index.html>
- 63 <https://docs.temporal.io/evaluate/use-cases-design-patterns>
<https://docs.temporal.io/evaluate/use-cases-design-patterns>
- 64 <https://docs.temporal.io/develop/python/best-practices/error-handling>
<https://docs.temporal.io/develop/python/best-practices/error-handling>
- 69 <https://docs.n8n.io/>
<https://docs.n8n.io/>
- 74 <https://microsoft.github.io/autogen/stable//index.html>
<https://microsoft.github.io/autogen/stable//index.html>
- 77 <https://docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.toolaiagent/>
<https://docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.toolaiagent/>
- 78 <https://arxiv.org/abs/2308.08155>
<https://arxiv.org/abs/2308.08155>
- 80 <https://cloud.tencent.com/document/product/1759/122549>
<https://cloud.tencent.com/document/product/1759/122549>
- 81 <https://www.aliyun.com/product/news/27058>
<https://www.aliyun.com/product/news/27058>
- 82 <https://cloud.tencent.com/developer/article/2520366>
<https://cloud.tencent.com/developer/article/2520366>
- 85 <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html>
<https://docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html>
- 87 <https://docs.temporal.io/self-hosted-guide>
<https://docs.temporal.io/self-hosted-guide>